

Loading Two Railroad Cars

David Bailey
Andrew Fedorchek
Larry Schauer
U.S. Military Academy
West Point, NY 10996-1786
Advisor: R.W. Sinclair

Abstract

This paper discusses the optimal way of loading crates of the same length and width, but of varying thicknesses and weights, onto two railroad flatcars, subject to constraints of weight, length, and local trucking regulations. The objective is to minimize the unused or wasted floor space on the rail cars. First, we compare the plane-cutting and enumeration techniques of solving integer linear programs. We decide on an enumeration technique known as *Krolak's bounded variable method*, which reduces the set of feasible solutions by progressively restricting the ranges of the variables. We then significantly reduce the number of combinations that must be evaluated by observing that using less than the maximum allotment of the first four crates cannot produce an optimal solution. Next, we combine these concepts into an algorithm, which we use to find the problem's optimal solutions.

"...combined
minimum
wasted floor
space of 6.0
mm produced
by 35 distinct
solution sets.."

We find a combined minimum wasted floor space of 6.0 mm between the two cars. This same optimal value is produced by 35 distinct solution sets. In addition to our solution, we include an analysis of the validity of our assumptions. We discuss the strengths of our model, including its speed and completeness. We also include an error and sensitivity analysis.

Problem Statement

The problem that we have been given is to find the optimal method of loading crates of given weights, thicknesses, and availabilities. The specifications of the crates are in Table 1.

Restrictions are imposed on the loading process by the physical limitations of the rail cars. Each car only has 10.2 m of available floor space and can only support 40 metric tons. Also, local trucking

Table 1

Crate types	1	2	3	4	5	6	7	
Thickness (t)	48.7	52.0	61.3	72.0	48.7	52.0	64.0	(cm)
Weight (w)	2	3	1	.5	4	2	1	(metric ton)
Number of crates available	8	7	9	6	6	4	8	

regulations limit the amount of space that can be occupied by crates 5, 6, and 7 to 302.7 cm.

Assumptions

- 1) The crates have no space between them.
- 2) The measurements that we have been given are perfect and contain no inherent error.
- 3) The crates cannot be divided, cut, or partitioned into smaller pieces.
- 4) No part of any crate can hang over the end of the rail car.
- 5) Any solution that we determine can actually be loaded and unloaded.
- 6) We do not need to use every type of crate.

Problem Justification

This problem lends itself to linear programming, since all of the constraints and the objective function are linear, and the constraints are inequalities. The simplex method is the most widely accepted technique for solving linear optimization problems. This method finds real solutions; but we require an integer solution, because the crates cannot be cut or divided. For this reason, we formulate the problem as an integer linear programming problem.

Definition of Terms

x_{ij} = number of crates of type i loaded on car j

$i \in \{1, 2, 3, 4, 5, 6, 7\}, j \in \{1, 2\}$

N_i = number of crates available of type i

w_i = weight of a crate of type i

t_i = thickness of a crate of type i

S = wasted space

Problem Formulation

As an integer linear programming problem, the model can best be expressed as follows:

MINIMIZE S , where

$$S = \left(10.2 - \sum_{i=1}^7 t_i x_{i,1} \right) + \left(10.2 - \sum_{i=1}^7 t_i x_{i,2} \right)$$

$$= 20.4 - \sum_{i=1}^7 t_i (x_{i,1} + x_{i,2})$$

subject to

$$x_{ij} \geq 0$$

x_{ij} is an integer

$$t_i x_i \leq 1020$$

$$w_i x_i \leq 40000$$

$$x_{i,1} + x_{i,2} \leq N_i$$

$$t_3(x_{3,1} + x_{3,2}) + t_6(x_{6,1} + x_{6,2}) + t_7(x_{7,1} + x_{7,2}) \leq 302.7$$

The two primary methods of solving integer linear problems are *plane-cutting* and *enumeration*. A comparison between these two types of methods shows that enumeration is more suitable for our needs. Plane-cutting does not yield any integer solution until it reaches an optimal solution, thereby disallowing the acceptance of a "reasonably good" solution. Also, plane-cutting stops at the first optimal integer solution reached. It will not continue to search for other solutions that are equally correct. Enumeration, on the other hand, will continually give a feasible solution while searching for a better solution. Additionally, enumeration techniques are appropriate for problems with fewer than 1000 variables. Since a finite number of feasible integer solutions exist, we could conceivably try every possible combination (but there are 2,520,473,760,000 of them). Instead, we develop a method to eliminate combinations that are obviously not feasible or optimal. This method is a form of enumeration known as the *bounded variable method*.

Our algorithm for reducing the set of values that we are required to evaluate hinges on several critical features of the problem. First, each of the variables is bounded both above and below. The upper bound for each variable is the maximum number of crates, and the lower bound for each variable is zero. Second, the objective function is formulated as a minimization problem but may just as easily be reformulated as a maximization problem. The problem is expressed in a form where the coefficients of the objective function and all the coefficients of the constraint matrix are nonnegative. This allows us to use a simplification of Krolak's bounded variable algorithm to reduce the feasible interval for each $x_{i,j}$.

Our evaluation of the variables is done with a branch tree, with one level for each variable. The concept of branching involves systematically moving from one possible solution set to another. Our evaluation starts with $x_{1,1}$ and proceeds to $x_{1,2}$, then to $x_{2,1}$, and so on, to $x_{7,2}$ on level 14. For convenience of notation and clarity of explanation, we rename the variables by their branch level: $x_{i,j} = x_{(i-1)*2+j}$, i.e., $x_{1,1} = y_1$; $x_{5,2} = y_{10}$; etc. Among all vertices emanating from a common node (or among all the successors emanating from a predecessor), the successors are examined in decreasing order of their value for the particular y_i . Once a vertex has been completely evaluated to the lowest level, we backtrack until we reach a vertex that has an unexamined successor, as shown in Figure 1.

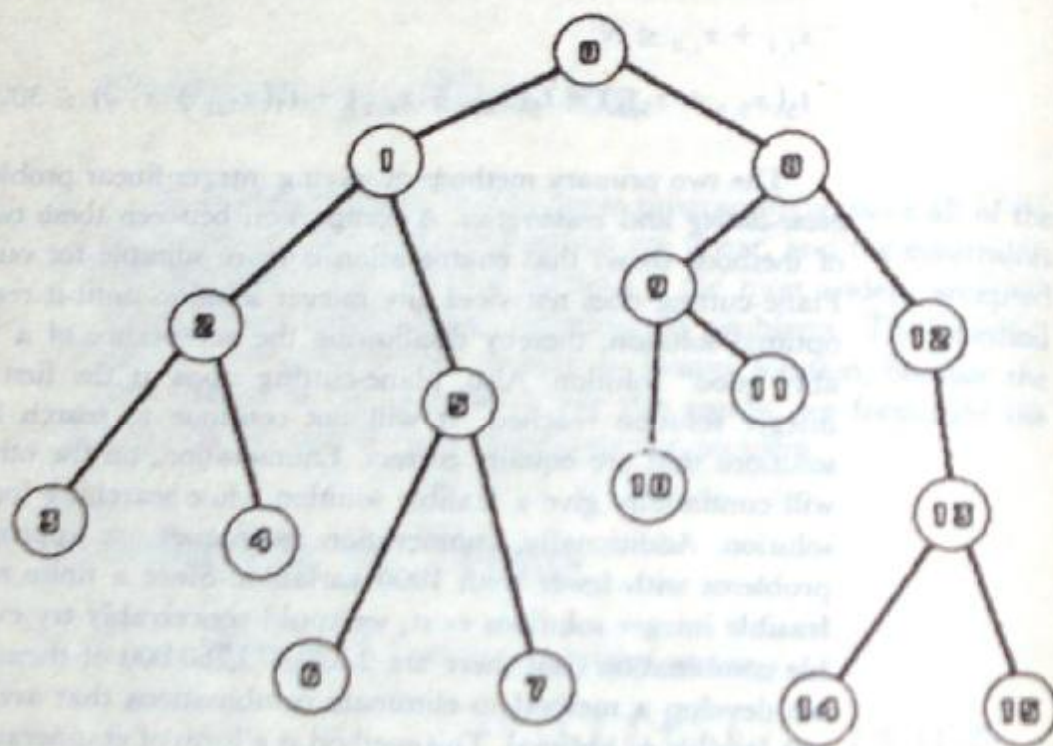


Figure 1. The vertices in this diagram illustrate the order in which vertices would be examined by the Krolak algorithm. The actual tree diagram for the rail car problem is much more complex.

Krolak's algorithm is now implemented to reduce the number of successors of a given vertex to be considered. Assume that we are selecting successors of a particular y_i . This means that along this branch the variables y_1 to y_i are constant. The variables y_{i+2} to y_{14} are now free to assume any value in their respective ranges. If we disregard the trucking regulation for the moment, we have three constraint equations in which y_{i+1} has nonzero coefficients: length, weight, and number of crates. Taking any arbitrary constraint

$$\sum_{k=1}^{14} a_k y_k \leq b_k,$$

we can write it in the form

$$\sum_{k=1}^i a_k y_k + a_{i+1} y_{i+1} + \sum_{k=i+2}^{14} a_k y_k \leq b_k,$$

where the first term is constant over the entire branch. Therefore, the constraint for the variable y_{i+1} is

$$y_{i+1} \leq \frac{\left[b_k - \sum_{k=1}^i a_k y_k - \sum_{k=i+2}^{14} a_k y_k \right]}{a_{i+1}}.$$

Since all $a_k > 0$, the maximum value for y_{i+1} is obtained by setting $y_k = 0$, for $k > i + 1$. This yields

$$y_{i+1} \leq \frac{\left[b_k - \sum_{k=1}^i a_k y_k \right]}{a_{i+1}}.$$

The three different constraints will then each set an upper bound on y_{i+1} . The maximum value of the variable will be the greatest integer not exceeding the minimum of the three bounds. If y_{i+1} corresponds to crates 5, 6, or 7, then an additional bound is calculated from the trucking constraint, and the minimum of the four is taken.

When $i = 1$, the bounds are of the form b_k/a_1 , which is positive; therefore $y_1 > 0$. For all subsequent variables, the preceding y_k will have been chosen so that the constraints are met, so

$$b_k - \sum_{k=1}^i a_k y_k \geq 0,$$

and therefore all $y_k \geq 0$. Our selection procedure is to calculate the

upper bound for each y and allow it to range from there to a lower bound of zero.

We will continue to do this until we reach level 14 of the tree, where an objective function is calculated. This value, Z , is compared to the upper and lower bounds. The initial upper and lower bounds are found from solving the associated continuous linear programming problems (the same problem formulation without restricting $X_{i,j}$ to the set of integers), first by maximizing the objective function and then by minimizing it. The first problem was done using the simplex method on a computer, and the maximum value was 2040 cm, i.e., no space was wasted. Finding the minimal value does not require any work, since total length used has a minimum of zero.

For a given value of Z , we first check to see if $Z > Z_{\text{lower}}$. If so, we let Z be the new lower bound and record the point. Note that whether Z exceeds the lower bound or not, we do not need to consider any other values of y_{14} . Decrementing y_{14} while holding all other variables constant only decreases the value of the objective function, which will definitely be less than the lower bound. The comparison to the upper bound is made to see if an optimal solution has been obtained, which would mean $Z = Z_{\text{max}}$. The search could stop at this point. We chose to continue in our algorithm, because we wanted to find all combinations of crates which would produce an optimal solution.

We have produced a general algorithm to solve this type of problem, but for our problem a relationship exists that greatly simplified the algorithm. If we were to use all of the available crates of the first four types, they would use 1737.3 cm of floor space. This would leave 302.7 cm of floor space for crates of types 5, 6, and 7, which coincides with the maximum allowed due to the local trucking restriction. Therefore, we are forced to use all of the crates of types 1, 2, 3, and 4 in order to minimize the wasted space. Not using all of those would leave unused space that could not be filled by crates of the other types because of the trucking restriction. This means that once a value for $x_{1,1}$ is chosen, the value for $x_{1,2}$ can be fixed at its upper bound. The number of available crates is the binding constraint in this case. This simplification reduces the size of the enumeration space by several orders of magnitude.

Finally, the number of points to be considered can be reduced by 50% by observing the symmetry of the problem. A solution for loading crates onto the cars will still be optimal if the cars are switched. Therefore the search for optimal points may be started at the midpoint for each of the variables, rounding up in the case of fractions. For ease of programming, the algorithm is simply started at $x_{1,1} = 4$. An algorithm that employs this technique for our loading problem is included as Appendix 1 [omitted]. Our associated

FORTRAN program is included as Appendix 2 [omitted]. The program finds all of the optimal solutions in less than 25 minutes on a personal computer.

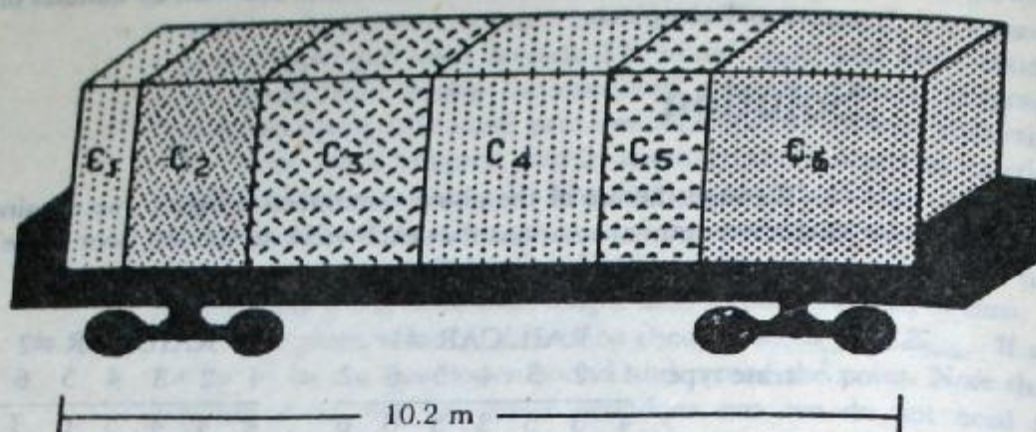
Solution

Through the use of the above mentioned program, we obtained 35 distinct optimal solutions that each yield an unused floor space of 0.6 cm. The solutions are as follows:

crate type	RAIL CAR #1							RAIL CAR #2						
	1	2	3	4	5	6	7	1	2	3	4	5	6	7
	4	3	5	3	3	0	0	4	4	4	3	0	3	0
	4	2	9	1	2	0	0	4	5	0	5	1	3	0
	4	2	5	3	3	1	0	4	5	4	3	0	2	0
	4	1	9	1	2	1	0	4	6	0	5	1	2	0
	4	1	5	3	3	2	0	4	6	4	3	0	1	0
	4	0	9	1	2	2	0	4	7	0	5	1	1	0
	4	0	5	3	3	3	0	4	7	4	3	0	0	0
	3	7	4	3	1	0	0	5	0	5	3	2	3	0
	3	7	0	5	2	1	0	5	0	9	1	1	2	0
	3	6	4	3	1	1	0	5	1	5	3	2	2	0
	3	6	0	5	2	2	0	5	1	9	1	1	1	0
	3	5	4	3	1	2	0	5	2	5	3	2	1	0
	3	5	0	5	2	3	0	5	2	9	1	1	0	0
	3	4	4	3	1	3	0	5	3	5	3	2	0	0
	3	2	9	1	3	0	0	5	5	0	5	0	3	0
	3	1	9	1	3	1	0	5	6	0	5	0	2	0
	3	0	9	1	3	2	0	5	7	0	5	0	1	0
	2	7	4	3	2	0	0	6	0	5	3	1	3	0
	2	6	4	3	2	1	0	6	1	5	3	1	2	0
	2	5	4	3	2	2	0	6	2	5	3	1	1	0
	2	5	0	5	3	3	0	6	2	9	1	0	0	0
	2	4	4	3	2	3	0	6	3	5	3	1	0	0
	1	6	4	3	3	1	0	7	1	5	3	0	2	0
	1	5	4	3	3	2	0	7	2	5	3	0	1	0
	1	4	4	3	3	3	0	7	3	5	3	0	0	0
	0	7	9	0	0	2	0	8	0	0	6	3	1	0
	0	7	6	4	0	0	0	8	0	3	2	3	3	0
	0	6	9	0	0	3	0	8	1	0	6	3	0	0
	0	6	6	4	0	1	0	8	1	3	2	3	2	0
	0	5	6	4	0	2	0	8	2	3	2	3	1	0

Note: The solution set is doubled if the cars are switched.

sample solution



Model Analysis

One of the first steps that we need to take in checking our model is error analysis. We need to check to see if our solution plus error falls within our constraints. While all of the constraints are important, the most binding constraint seems to be the length restriction. Therefore we are going to show the error analysis of the length to a 95% confidence level. Error analysis for the other constraints will be similar. The equation that we will use is *

$$(\text{error per measurement})(\sqrt{n}) = \text{error at 95\% confidence}$$

The measurements are accurate to 0.05 cm, and there are $n = 36$ crates loaded in each optimal solution. From this we can see that the error at 95% is 0.3 cm. Because this value falls within our acceptable region, we can conclude that our assumption that the measurements are exact is valid. Unfortunately, the error is divided between the two cars.

We must also question the assumption that there is no extra space between the crates. This assumption is necessary to form a mathematical model but is inaccurate in reality. There will be a certain amount of debris between the crates due to dirt, etc. Therefore, this assumption compromises the integrity of our model. The other assumptions are physical restraints and do not alter the validity of the model.

The next logical step in analyzing our model is a sensitivity analysis. A thorough sensitivity analysis covers several possible changes. These changes include changing the objective, changing the

crate specification, changing the physical restrictions, adding/subtracting a crate type, or adding/subtracting a physical restraint. Any one of these changes would result in a new problem that would need to be solved again. In fact, a change in any of the constraints could force us to evaluate a larger number of the feasible solutions. If the local trucking restriction did not exist, it would be possible to obtain a solution that resulted in zero wasted space. One such solution for example would be

car 1: 2 6 2 5 0 0 2 car 2: 6 1 6 0 0 1 4

A reduction in the allowable weight for each car would eliminate feasible solutions; while if the weights of any of crates of types 1 through 6 would decrease, the number of optimal solutions could increase. Crates of type 7 are unique in that a change in their weight or availability would have no effect on the number of optimal solutions. A change in the width of crates of type 7, however, could increase the number of optimal solutions.

Strengths and Weaknesses

Our algorithm features several important strengths. First, it rapidly obtains a feasible integer solution with a corresponding value of the objective function. Even if this is not necessarily the optimal solution, it is a solution. For this particular problem, we obtained the value of 0.6 cm for wasted space after 8 seconds. Second, as the branch is entered, the limit of a given variable is calculated so that all of the constraint equations are satisfied. This means that the variable is allowed to range from this limit to zero without checking to see if the constraints are met, thus saving time. Third, by starting from the largest value and working down on y_{14} , only one value for y_{14} is ever considered for a given set of values of y_1 through y_{13} .

Unfortunately, while the general algorithm can easily be adapted to changes in the problem formulation, our special-case algorithm is not nearly as flexible.

References

- Cooper, Leon, and David Steinberg. 1970. *Introduction to Methods of Optimization*. Philadelphia, PA: W.B. Saunders.
- Garfinkel, Robert S., and George L. Nemhauser. 1972. *Integer Programming*. New York: John Wiley & Sons.
- Rao, S.S. 1978. *Optimization Theory and Applications*. New Delhi, India: Wiley Eastern Limited.